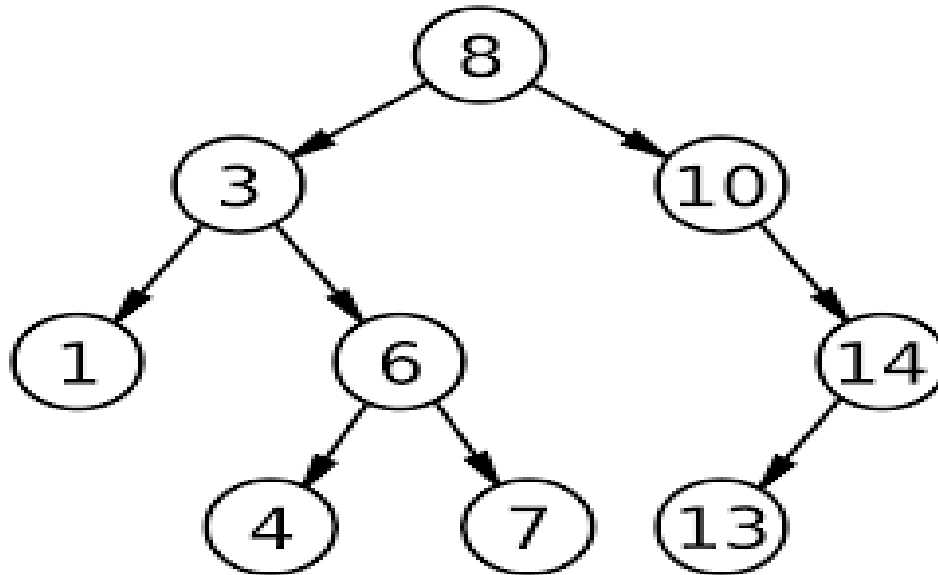


Binary Tree

Binary trees

- A binary tree is an ordered tree having at most two nodes
- Node at left side of a node is called left node and at right side is called right node
- No nodes can have more than two subtree
- Subtrees are designed as left and right subtree.



Complete Binary tree

- Every nodes (except leaf) having two nodes
- Maximum number of nodes in Level i is 2^i
nodes in level $0 = 2^0 = 1$, level $1 = 2^1 = 2$

- **In a tree of height, h**

- Leaf/leaves are at level h is 2^h

- Number of nodes n ,
 $= 2^0 + 2^1 + 2^2 + \dots + 2^h$
 $= 2^{h+1} - 1$

$$n = 2^{\text{levels}} - 1$$

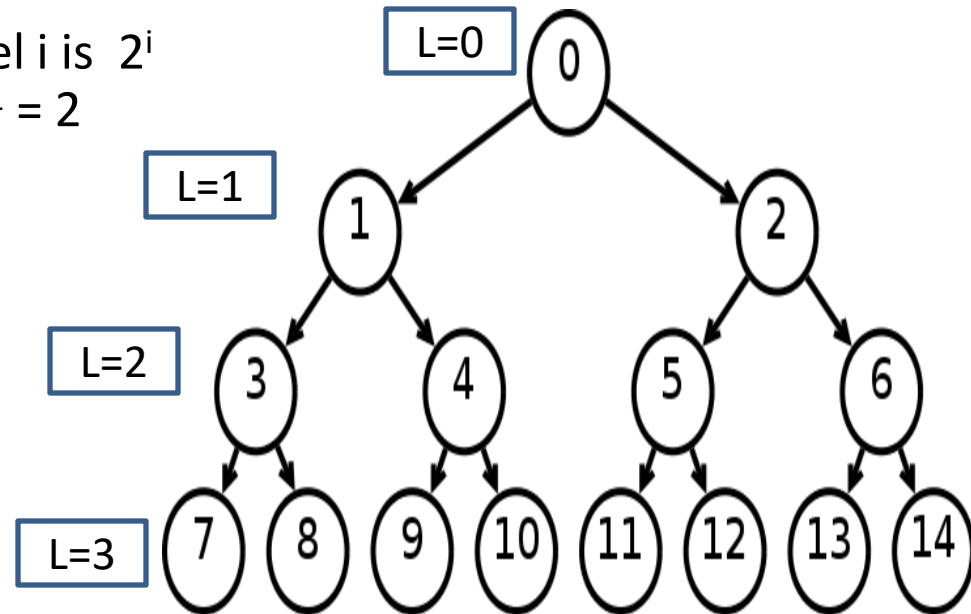
- **In a tree of node n ,**

- The height h is : $n = 2^{\text{levels}} - 1$

$$\Rightarrow n+1 = 2^{\text{levels}} = 2^{h+1}$$

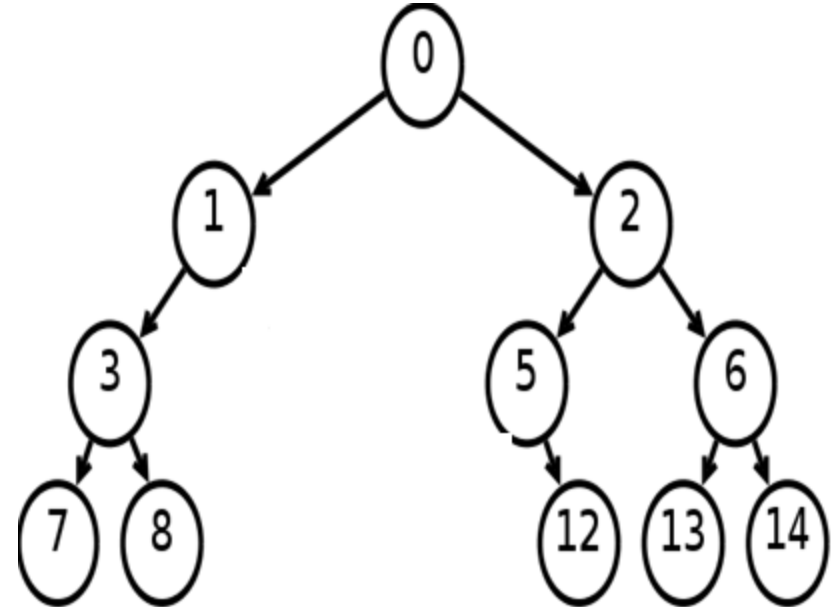
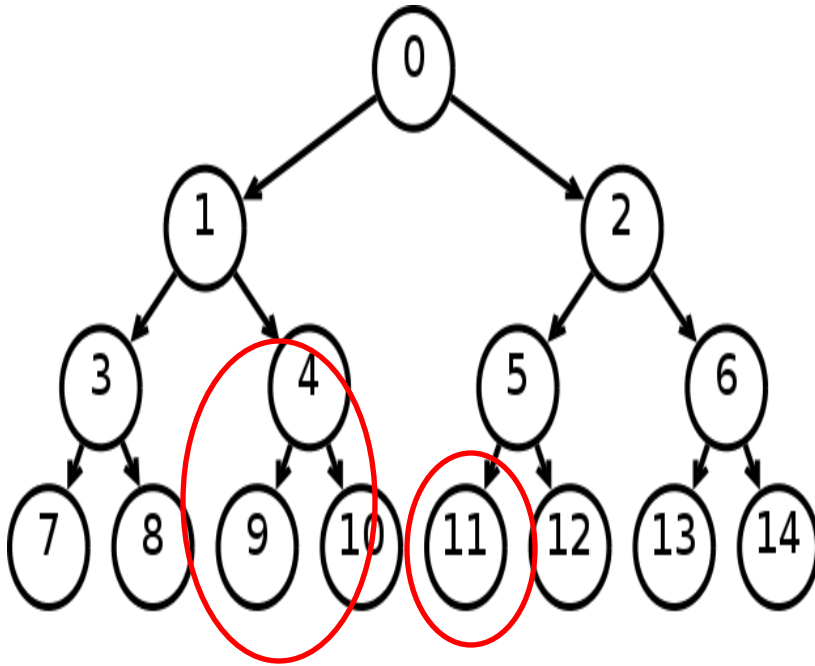
$$\Rightarrow \log_2 (n+1) = h+1$$

$$\Rightarrow h = \log_2 (n+1) - 1$$



Binary tree from complete binary tree

- A binary tree can be obtained from a complete binary tree by pruning

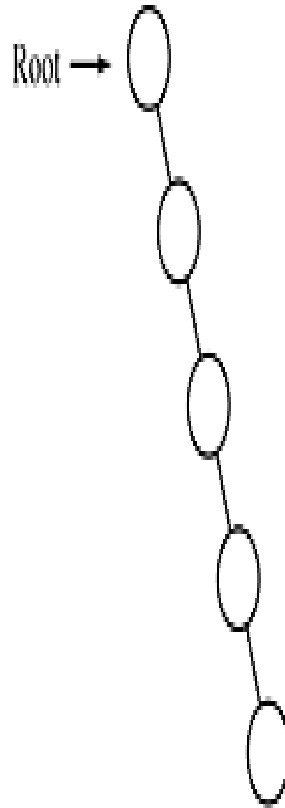


Minimum height of a Binary trees

- **A binary tree of height h has,**
 - At most 2^i nodes in level i
 - At most $2^0 + 2^1 + 2^2 + \dots + 2^h = 2^{h+1} - 1$ nodes
- **If the tree has n nodes then,**
 - $n \leq 2^{h+1} - 1$
 $\Rightarrow h \geq \log_2 (n+1) - 1$
- A binary tree has at least height of $\log_2 (n+1) - 1$ which is a complete binary tree

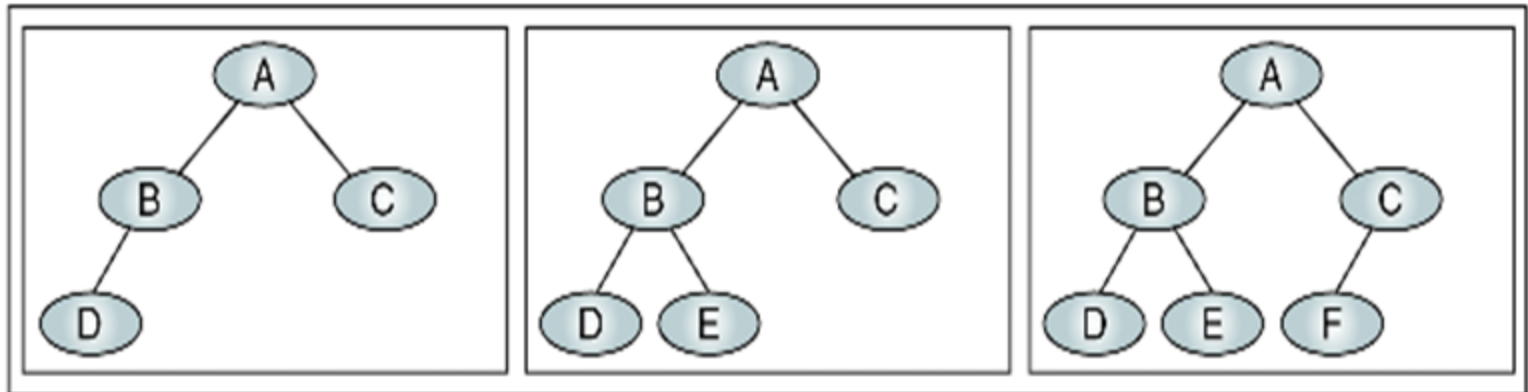
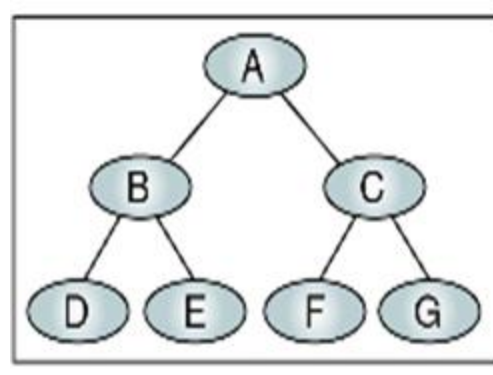
Maximum height of a Binary trees

- If the tree has n nodes then the height is $n-1$
- This is obtained when every node has exactly one child (except leaf node)



Near Complete Binary Tree

- If a tree has the minimum height for its nodes and all nodes in the last level are found on the left is called near complete binary tree.



Traversing Binary Tree

There are 3 ways of traversing a binary tree T having root R.

1. Preorder Traversing

- Steps:
 - (a) Process the root R
 - (b) Traverse the left subtree of R in preorder.
 - (c) Traverse the right subtree of R in preorder.
- Example:

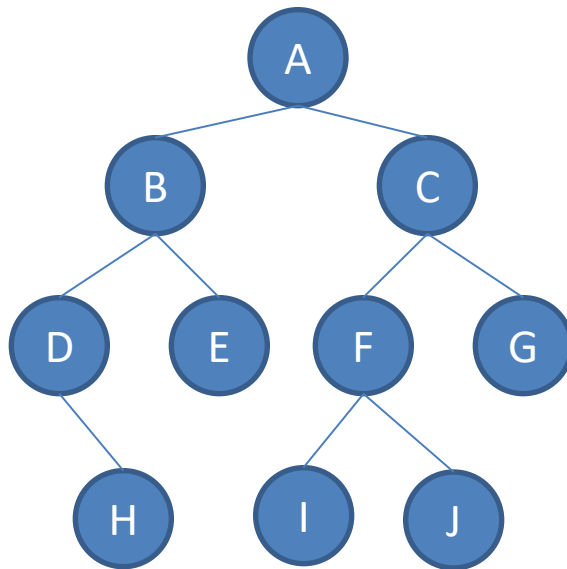


Figure: Binary Tree T

Preorder Traversal of T

A, B, D, H, E, C, F, I, J, G

2. Inorder Traversing

- Steps:
 - (a) Traverse the left subtree of R in inorder.
 - (b) Traverse the root R.
 - (c) Traverse the right subtree of R in inorder.
- Example:

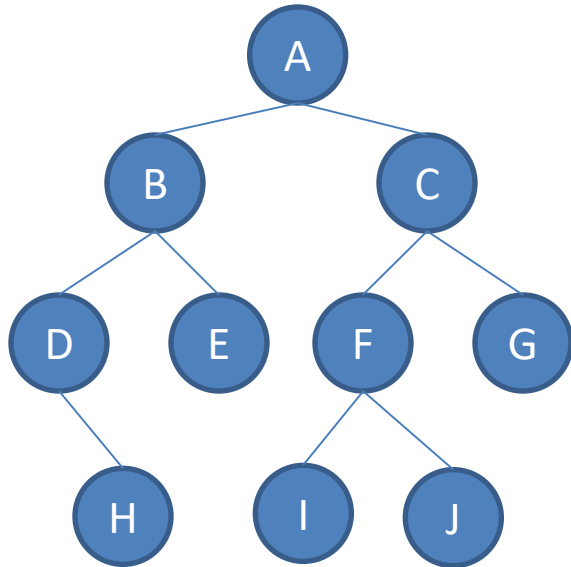


Figure: Binary Tree T

Inorder Traversal of T

D, H, B, E, A, I, F, J, C, G

3. Postorder Traversing

- Steps:
 - (a) Traverse the left subtree of R in postorder.
 - (b) Traverse the right subtree of R in postorder.
 - (c) Traverse the root R.
- Example:

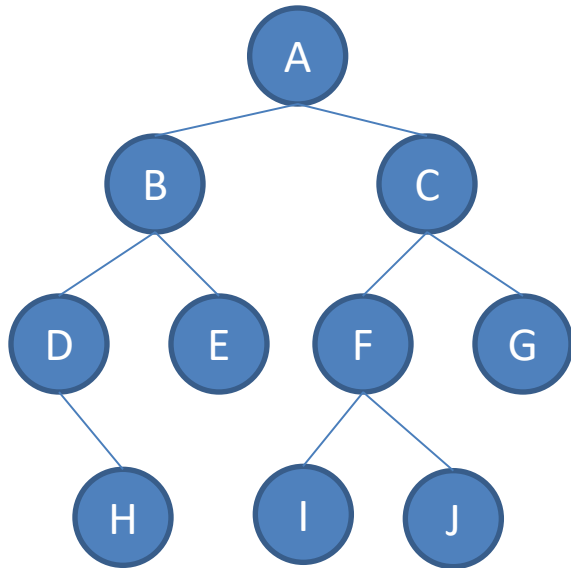


Figure: Binary Tree T

Postorder Traversal of T

H, D, E, B, I, J, F, G, C, A

Binary Search Trees

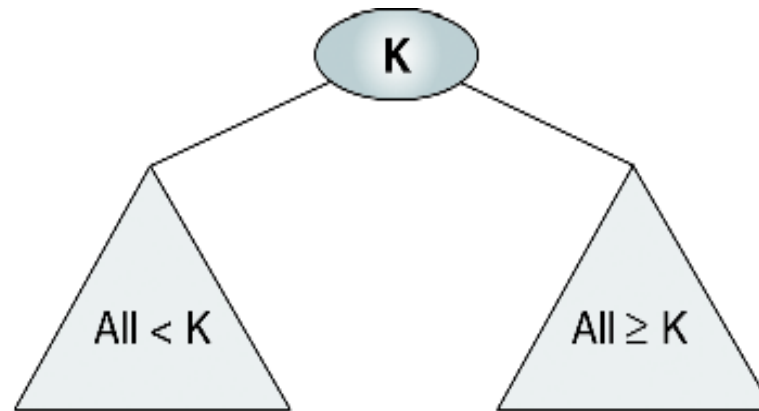
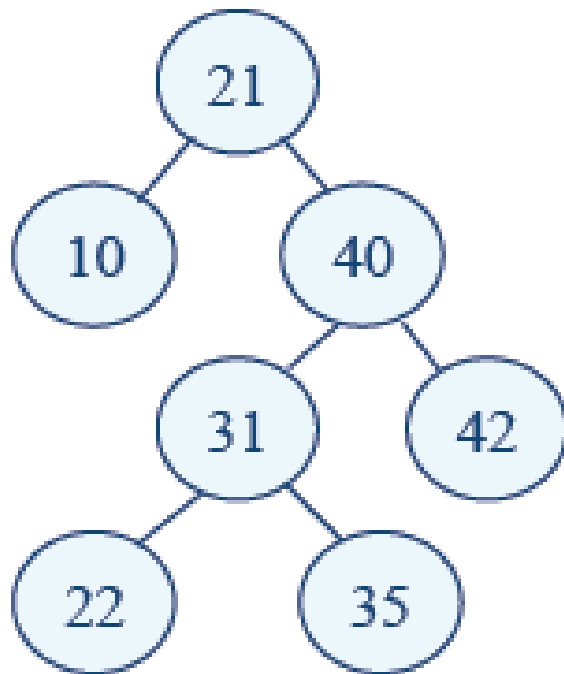


FIGURE 7-1 Binary Search Tree

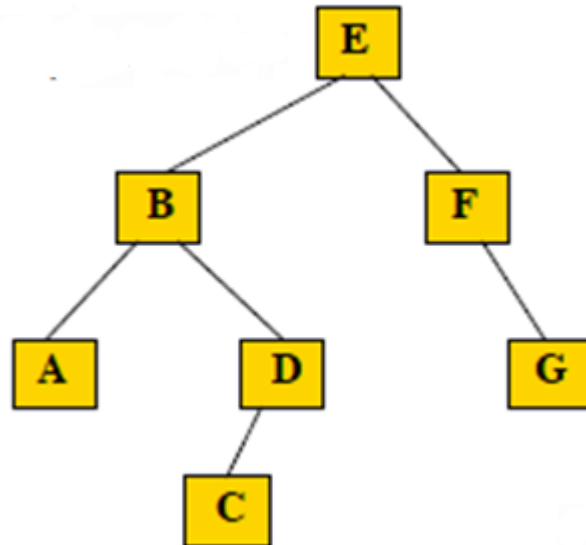
Binary search trees (Con..)



Binary Search Tree (Con..)

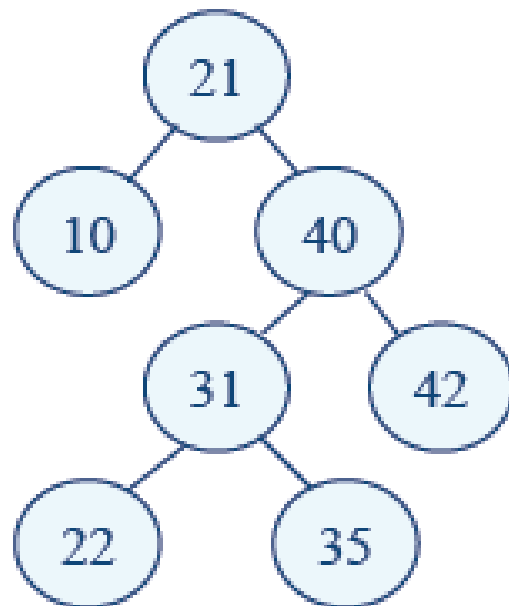
A **Binary Search Tree** is a binary tree with the following Basic properties:

- All items in the left subtree are less than the root.
- All items in the right subtree are greater or equal to the root.
- Each subtree is itself a binary search tree.



Binary search trees (Con..)

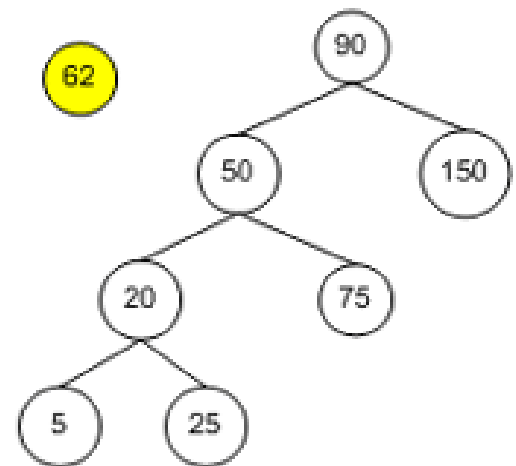
- If we want to find out whether a given object is present in a binary search tree:



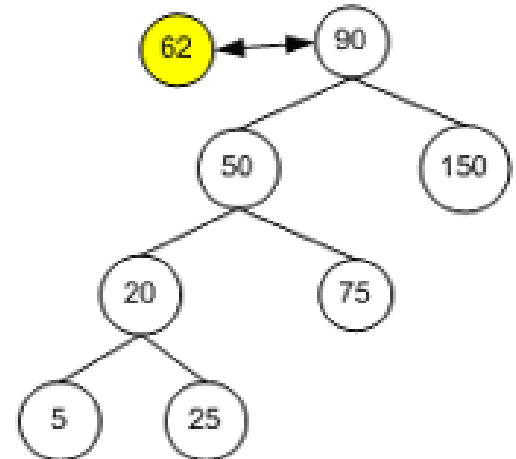
- Suppose the item is the number 39. Beginning at the root of the tree, we see that 39 is greater than 21.
- If 39 is in the tree, it must be in the right hand tree with respect to the root node.
- We see that 40 is greater than 39, so we turn to the left hand tree with respect to 40.
- We eventually establish that 40 is not in the tree.

Searching and inserting
in Binary search trees

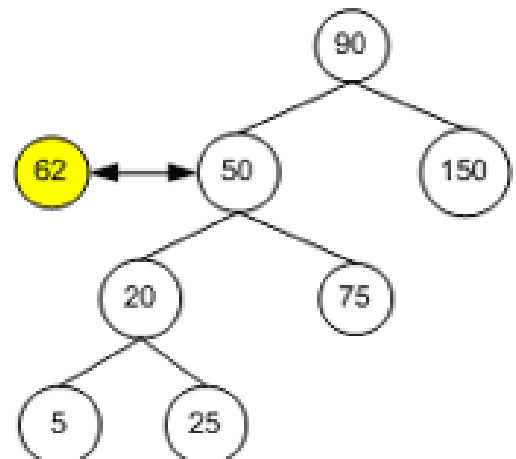
Given the following BST, we want to insert a node with the value 62...



We start by comparing the node to insert (62) with the root (90). We see that 62 is less than 90, so we know 62 must be added somewhere to the root's left subtree.

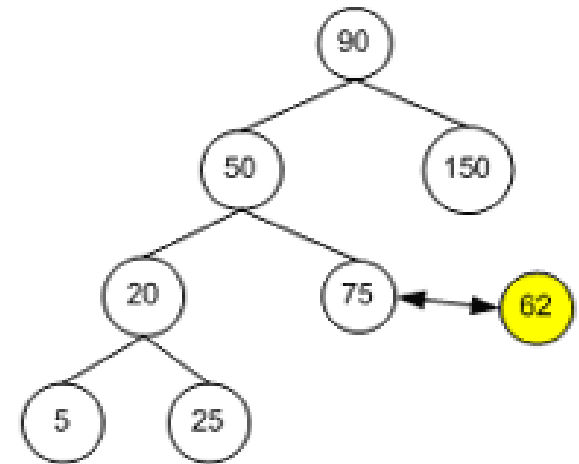


We next compare 62 to 50. Since 62 is greater than 50, 62 must belong somewhere in 50's right subtree.



Searching and inserting
in Binary search trees

We next compare 62 to 75.
Since 75 is greater than 62,
62 must exist somewhere in
75's left subtree.



Since 75's left child is a null
reference, we have found the
new location for node 62!

All that's left to do is set 75's
left child to 62, which adds 62
to the BST tree and maintains
the binary search tree
property.

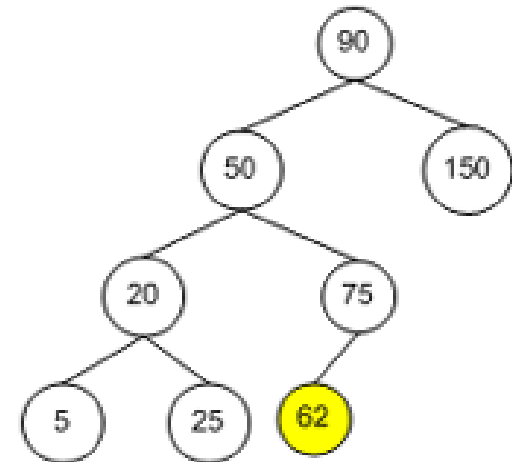
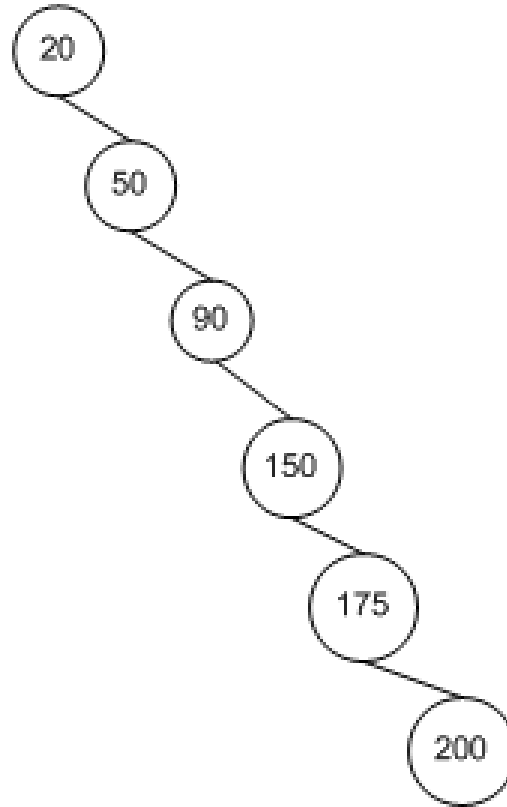


Figure 1. Inserting a new node into a BST

Searching and inserting in Binary search trees



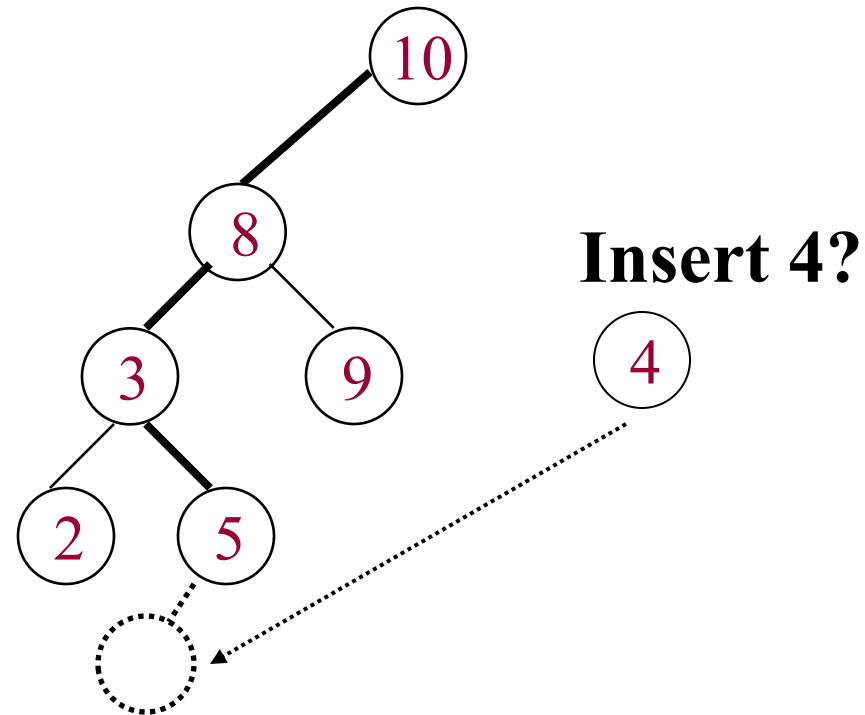
A BST after nodes with values of 20, 50, 90, 150, 175, and 200 have been added

Inserting a new key in a BST

How to insert a new key?

The same procedure used for search also applies: Determine the location by searching. Search will fail. Insert new key where the search failed.

Example:



Building a BST

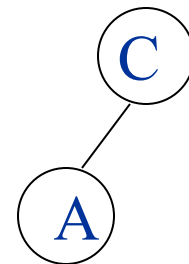
Build a BST from a sequence of nodes read one a time

Example: Inserting **C A B L M** (in this order!)

1) Insert C

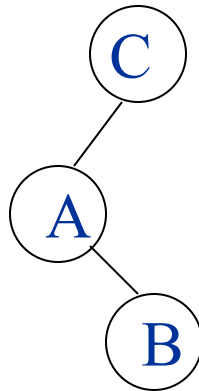


2) Insert A

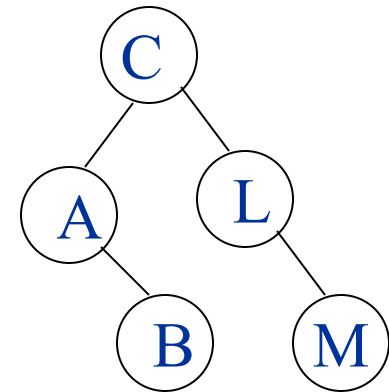


Building a BST

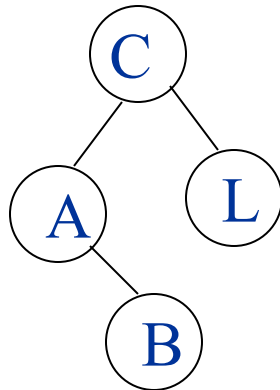
3) Insert B



5) Insert M



4) Insert L

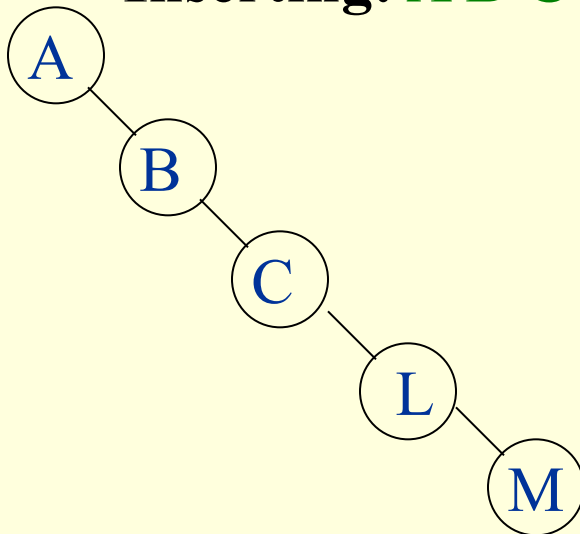


Building a BST

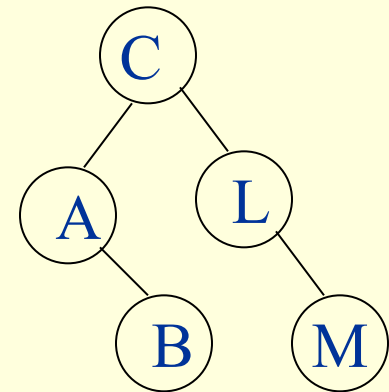
Is there a unique BST for letters A B C L M ?

NO! Different input sequences result in different trees

Inserting: **A B C L M**



Inserting: **C A B L M**



Sorting with a BST

Given a BST can you output its keys in sorted order?

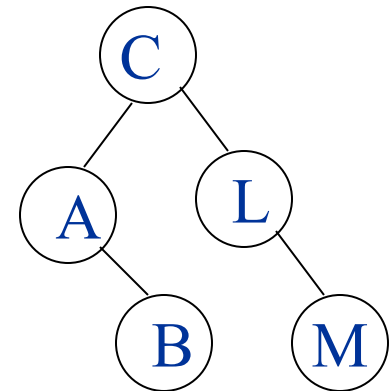
Visit keys with Inorder:

- visit left
- print root
- visit right

How can you find the minimum?

How can you find the maximum?

Example:



Inorder visit prints:

A B C L M

Preorder Traversal

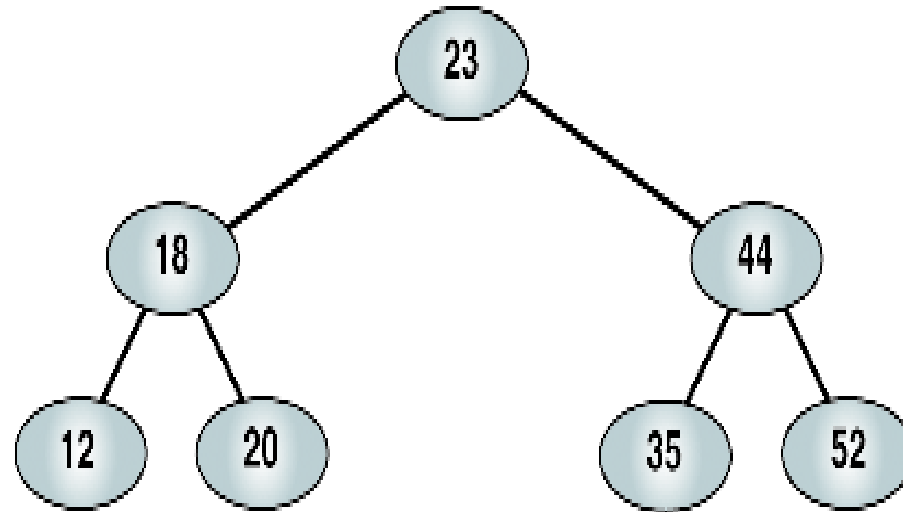


FIGURE 7-4 Example of a Binary Search Tree

23 18 12 20 44 35 52

Postorder Traversal

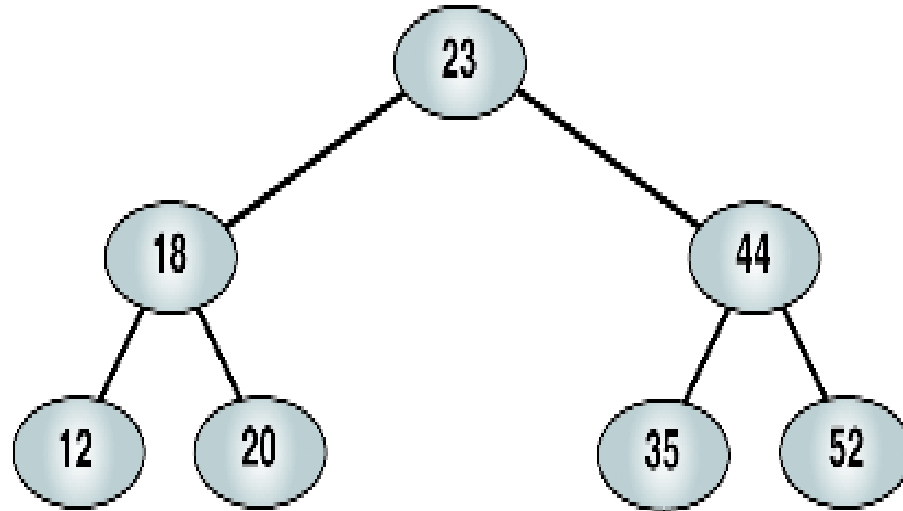


FIGURE 7-4 Example of a Binary Search Tree

12 20 18 35 52 44 23

Inorder Traversal

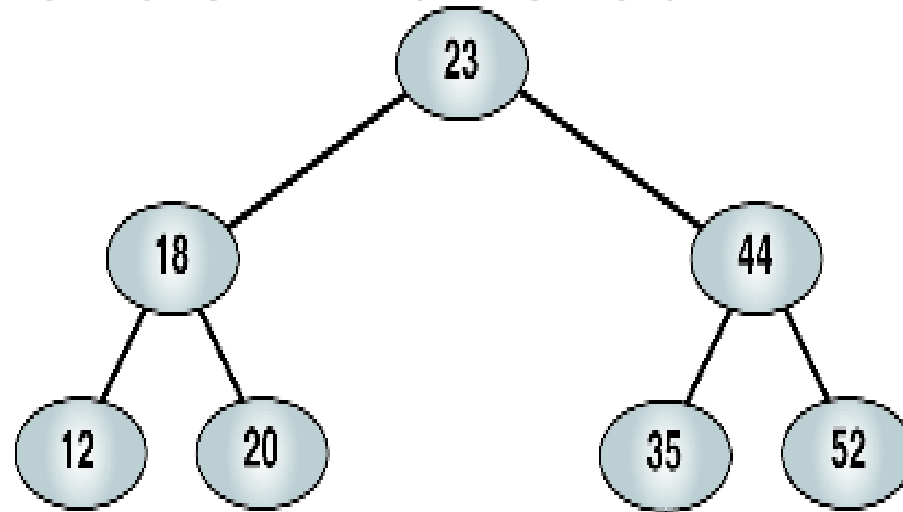


FIGURE 7-4 Example of a Binary Search Tree

12 18 20 23 35 44 52

Inorder traversal of a binary search tree produces a
sequenced list

Right-Node-Left Traversal

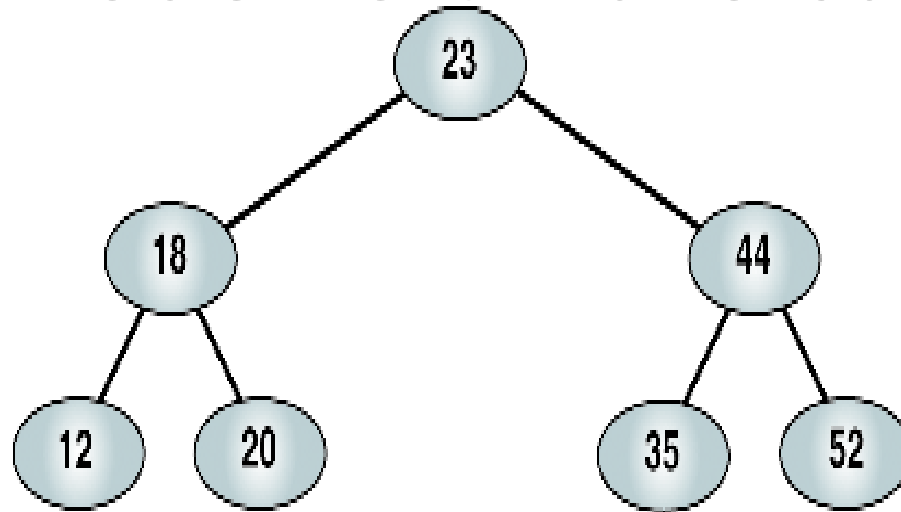


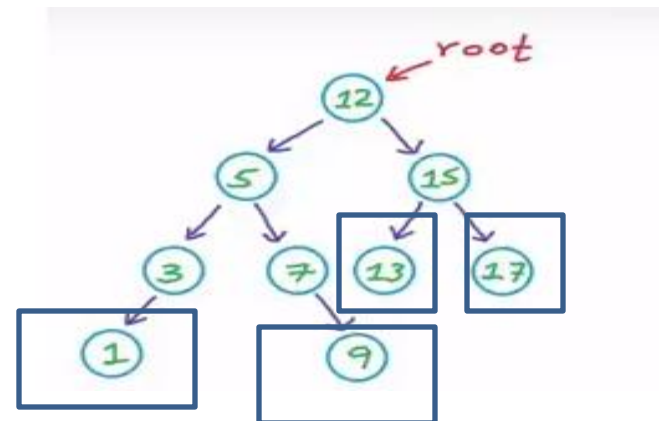
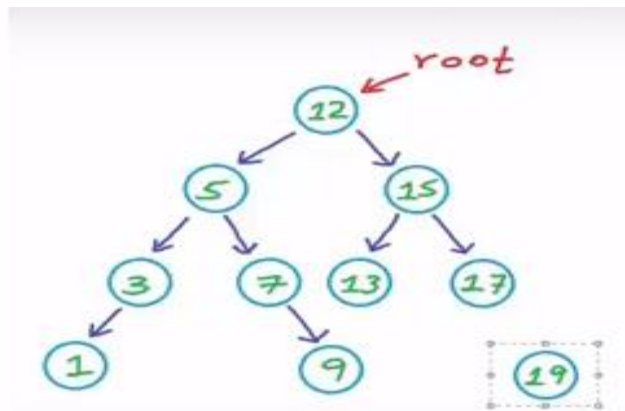
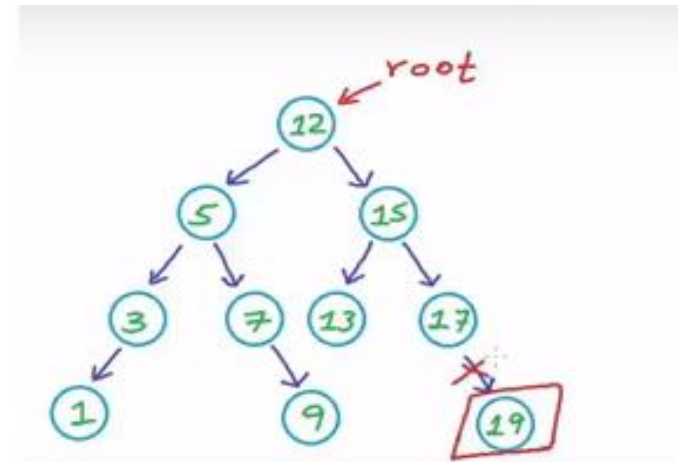
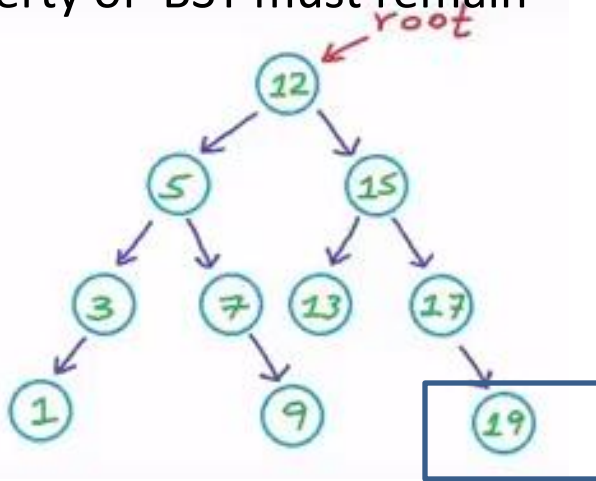
FIGURE 7-4 Example of a Binary Search Tree

52 44 35 23 20 18 12

Right-node-left traversal of a binary search tree produces a **descending sequence**

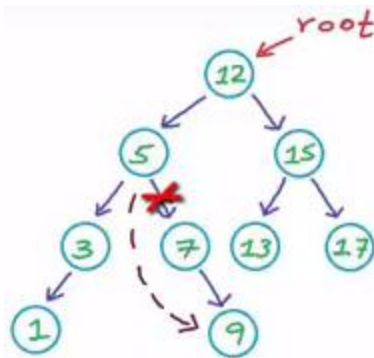
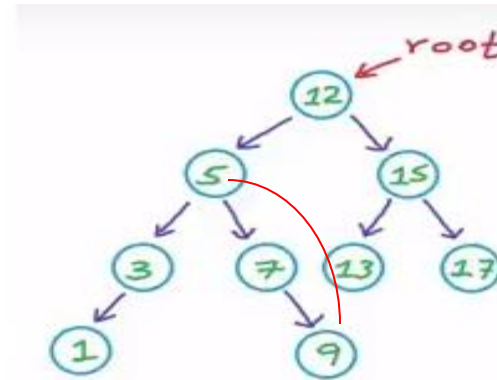
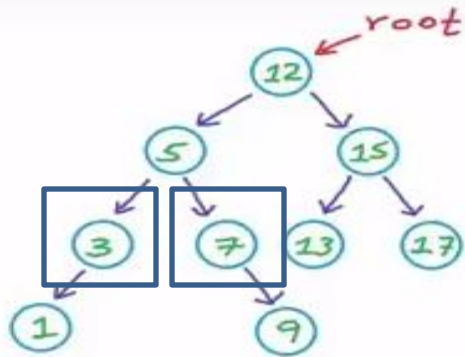
Delete a node from BST

- **Case 1:** leaf node has no child, so this node can easily wiped out from memory
- Property of BST must remain

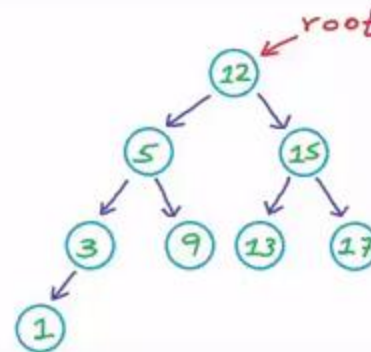


Delete a node from BST

- **Case 2:** if node has one child (left or right), then link the parent node with the child node and wipe out the node from memory
- Property of BST must remain

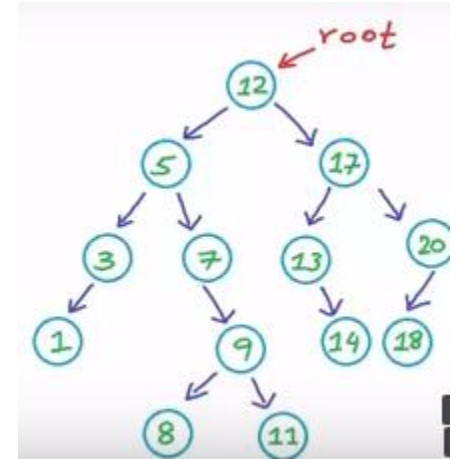
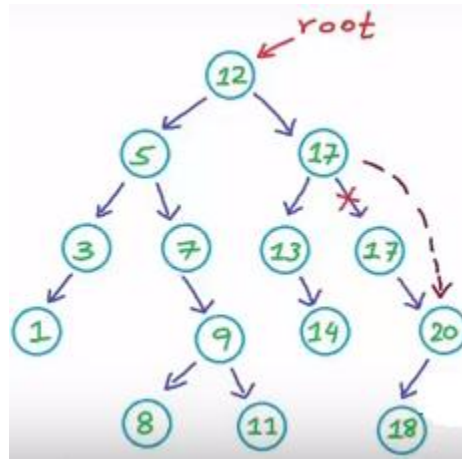
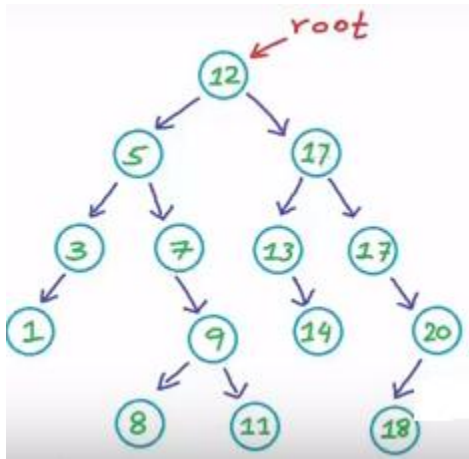
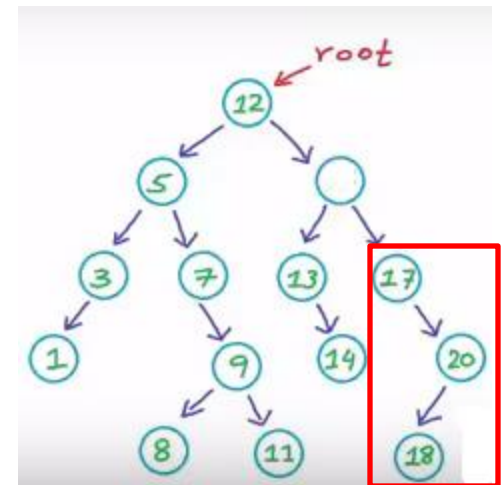
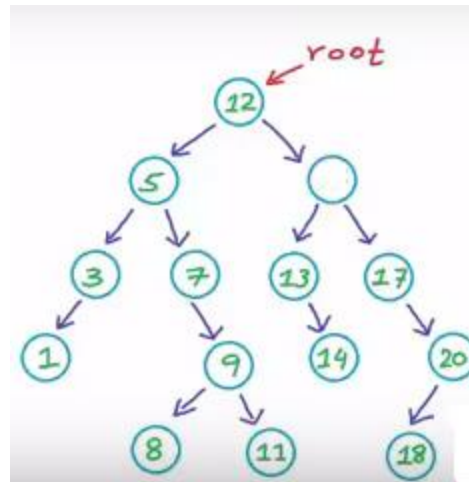
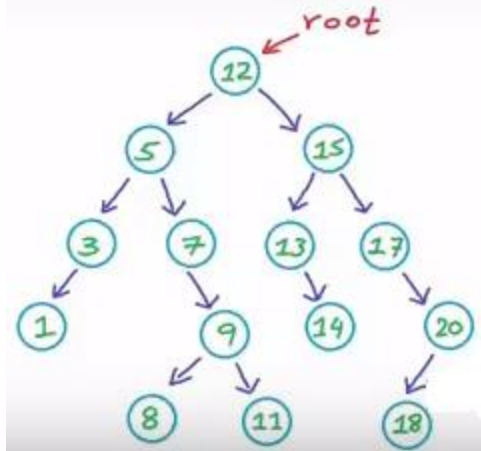


⇒



Delete a node from BST

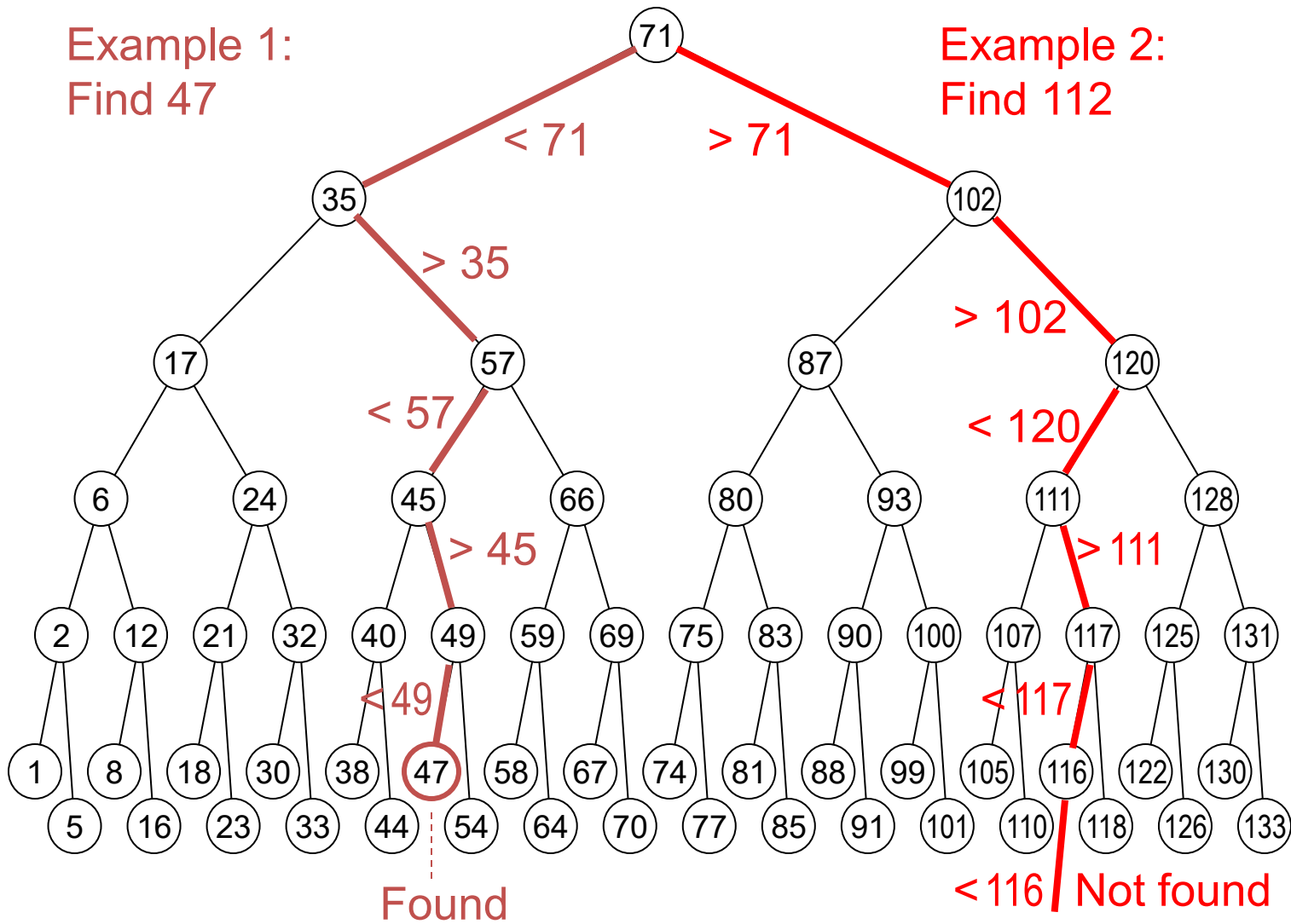
- **Case 3:** if node has two child then two case can be considered:
 - Find min node from right subtree or max node from left subtree
 - save the min value in the place of the node deleted
 - delete the duplicate value
- Property of BST must remain



Binary Search Trees

Example 1:
Find 47

Example 2:
Find 112

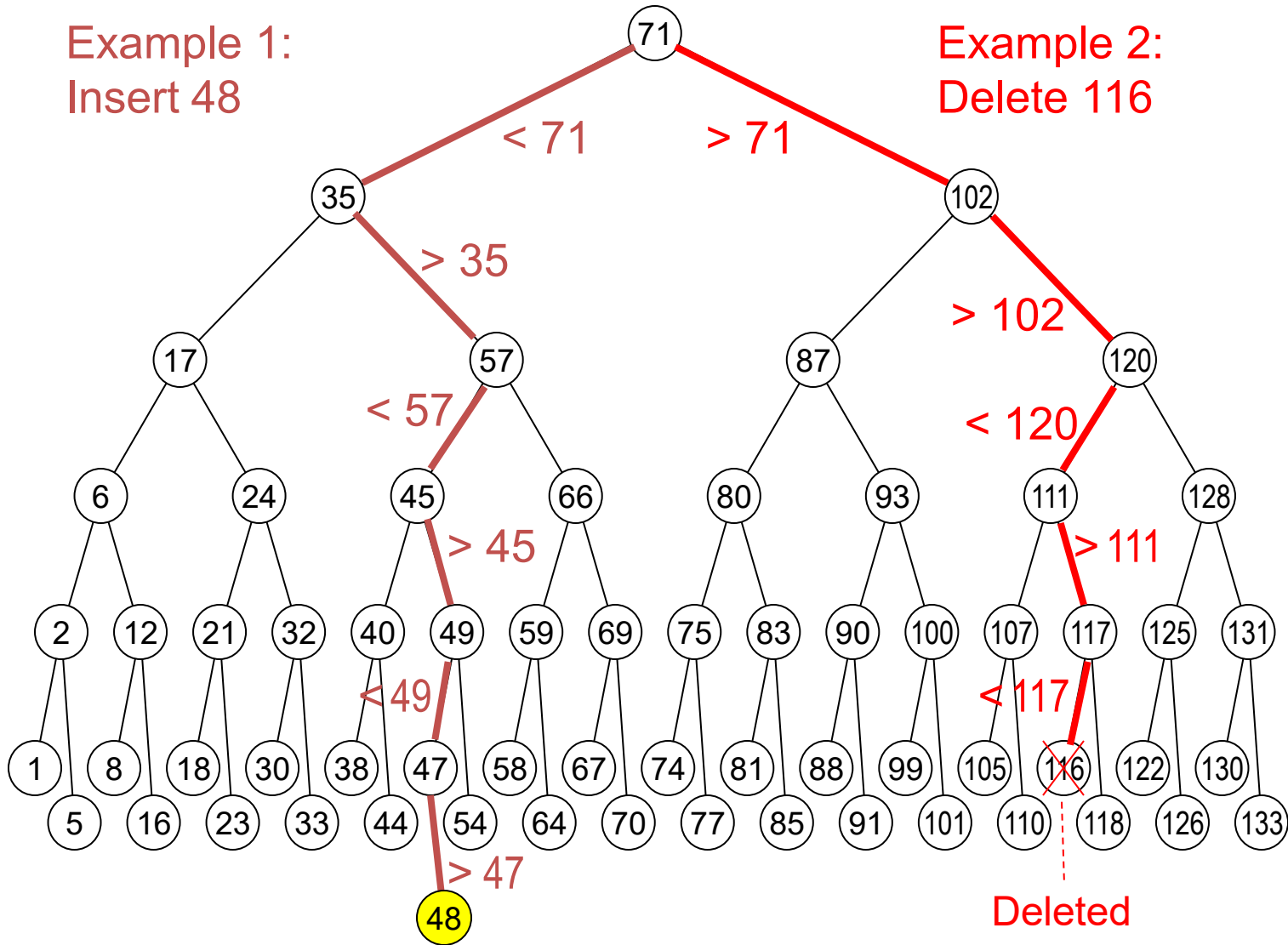


63-item list

Insertions and Deletions in Binary Search Trees

Example 1:
Insert 48

Example 2:
Delete 116



Exercises

- Chapter 6: Tree
Exercises: 10, 11, 13, 14, 15, 30
- Chapter 7 (BST)
Exercises: 2, 5, 9, 11

Source: Data Structures: A Pseudocode Approach Using C, 2nd ed